Creating an AI-Agent using Transformer Model and MCP to Provide Treatment/Diagnosis of Common Illnesses

Dennis Tislin

Computer Systems Lab

Dr. Yilmaz

5/20/2026

Table of Contents

Abstract

The purpose of my research project is to create a fully functional, maximal utility innovation in the form of an application of Artificial Intelligence. Within the field of healthcare, AI applications are uprising creations that continue to grow. Of the many applications, a considerable amount tend to focus on issues affecting small populations –cancers and rare illnesses. Even though research for such a cause has proven invaluable, its effect on the healthcare economy has been minimal. America's healthcare system experiences a tragic flaw of continuously increasing prices, struggling to provide affordable treatment for middle/low class citizens suffering illnesses. In 2023, Medicare spending grew 8.1% to $1,029.8 billion and Medicaid spending grew 7.9% to $871.7 billion [8]. Most of these individuals, however, face struggle with common illnesses rather than rare exceptions. In order to ameliorate healthcare costs, it is necessary to create an AI application in the form of an Agent that specializes in common illnesses. By creating a cost-efficient, reliable AI integrated alternative to common illness treatment and diagnosis, the flawed healthcare system can eventually be improved. Through the utilization of Large Language Models (LLM) on Hugging Face and fine tuning an optimized domain-specific model, an AI Agent consisting of a Model Context Protocol (MCP) Client, LLM, and MCP Server can replicate the basic tasks of medical workers who provide diagnosis, treatments, and prescription suggestions of common illnesses.

System Architecture

In order to create a cost-efficient, accessible resource for common illnesses, an AI Agent can be created through the incorporation of an LLM and MCP server, of which can be connected through an MCP client. Once created, such an application will be able to process user input in the form of text, simulating a conversational environment. Based on a user's input, the LLM will determine whether it must connect to the MCP server through the Client and conduct an action, or simply provide its own text output. Contingent on user request, the model connects to the server to gain access to tools that allow it to conduct actions and be able to successfully accomplish its goal. Specifically, by utilizing Transformer Models to process sequential patterns and understand the relationships of English in Natural Language Processing, an AI Agent connected to an MCP server can effectively diagnose the symptoms of common illnesses, suggest treatment, and contact medical providers for over the counter medicine/treatment. By processing user input into the model to predict the best output, the model provides an accurate diagnosis. On the contingent command, the Model determines if the MCP Client must connect to the server and execute an action from a pipeline consisting of tools. Figure 1 outlines this architecture.

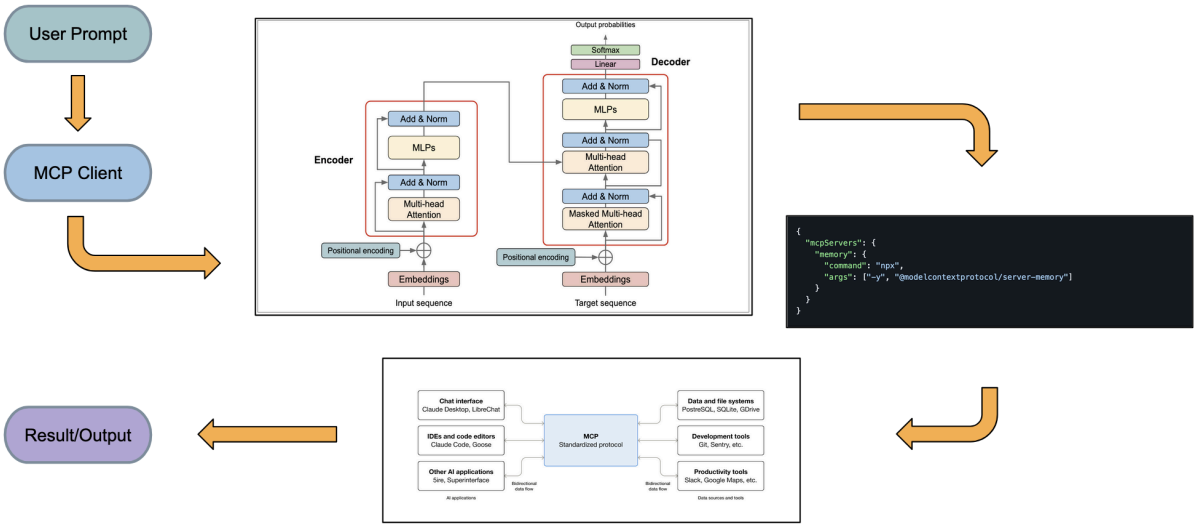

Figure 1

Dependencies

The initialization of this project depends on a broad range of libraries/dependencies that must be installed, in order to be able to create the components of an AI Agent. For organization purposes, these dependencies are installed within a Conda virtual environment. When initially finding the dataset of medical records to fine-tune the chosen model on, the open-source AI Community Kaggle was able to provide a Comma Separated Value (CSV) file consisting of common illnesses and their symptoms [16]. To be able to download and utilize this dataset, it is required to create a Kaggle API Access Token. After doing so, it becomes necessary to install the libraries needed to convert this Kaggle Disease to a Hugging Face dataset. The open source organization Hugging Face provides a myriad of models previously trained/fine-tuned, or to be fine tuned. In order to be able to fine tune an LLM from Hugging Face, it must be done through datasets a part of the Hugging Face Hub and loadable through the datasets library. The “huggingface-hub” library is needed to be installed through pip, the python package installer. Other than converting the

Kaggle dataset, fine tuning the LLM requires Pytorch, Hugging Face, and optimization libraries. Furthermore, creating the MCP server requires universal libraries along with libraries specifically for parsing web-pages, which is used within this project. All necessary dependencies are within the “requirements.txt” found in my GitHub repository [17].

Custom Fine Tuning

After setting up a Conda virtual environment with all required dependencies, the creation of the Agent may commence. When fine tuning an LLM from Hugging Face on a dataset from Kaggle, it is crucial to first convert the Kaggle dataset, otherwise it would not be possible; Hugging Face LLMs can only be trained/fine tuned on datasets within their database. With this, the conversion of the Kaggle dataset is done through an altered GitHub python script specifically for this process. However, the initial python script [11] was unable to convert the dataset because of an outdated API call within one of its functions: “'KaggleApi' object has no attribute 'metadata_get'.” The CSV dataframe that I am attempting to convert from Kaggle cannot have its data be processed/converted; the python function in the Kaggle API cannot do this action because of an error drawn within its code, an updated version that is no longer compatible, or the removal of the function entirely. After creating an alternative function that allows the script to successfully convert the Kaggle Dataset, it becomes possible to fine tune a chosen LLM. After being converted to a Hugging Face dataset, the Kaggle dataset went from initially being a CSV format file to a parquet file, of which is a Numpy array of lists of diseases and their symptoms, as shown in figure 2.

Disease and Symptoms on Kaggle

Disease Disease name	Symptom_1 Symptom of the disease	Symptom_2 Symptom of the disease	Symptom_3 Symptom of the disease
41 unique values	vomiting 17% fatigue 14% Other (3408) 69%	vomiting 18% fatigue 8% Other (3648) 74%	fatigue 15% high_fever 7% Other (3870) 79%
Fungal infection	itching	skin_rash	nodal_skin_eruptions
Fungal infection	skin_rash	nodal_skin_eruptions	dischromic _patches
Fungal infection	itching	nodal_skin_eruptions	dischromic _patches
Fungal infection	itching	skin_rash	dischromic _patches
Fungal infection	itching	skin_rash	nodal_skin_eruptions
Fungal infection	skin_rash	nodal_skin_eruptions	dischromic _patches
Fungal infection	itching	nodal_skin_eruptions	dischromic _patches
Fungal infection	itching	skin_rash	dischromic _patches
Fungal infection	itching	skin_rash	nodal_skin_eruptions
Fungal infection	itching	skin_rash	nodal_skin_eruptions

Example from Train split

disease string · classes 41 values	symptoms list · lengths 3 17
Fungal infection	["itching", "skin_rash", "nodal_skin_eruptions", "dischromic _patches"]
Psoriasis	["skin_rash", "joint_pain", "skin_peeling", "silver_like_dusting", "small_dents_in_nails",…
Alcoholic hepatitis	["vomiting", "yellowish_skin", "abdominal_pain", "swelling_of_stomach", "history_of_alcohol_consumption",…
Hypoglycemia	["vomiting", "fatigue", "anxiety", "sweating", "headache", "nausea", "blurred_and_distorted_vision",…
Pneumonia	["chills", "fatigue", "cough", "high_fever", "breathlessness", "sweating", "malaise", "phlegm",…
Typhoid	["chills", "vomiting", "fatigue", "high_fever", "headache", "nausea", "constipation", "abdominal_pain",…
Fungal infection	["itching", "skin_rash", "nodal_skin_eruptions", "dischromic _patches"]
Bronchial Asthma	["fatigue", "cough", "high_fever", "breathlessness", "family_history", "mucoid_sputum"]
Osteoarthristis	["joint_pain", "neck_pain", "knee_pain", "hip_joint_pain", "swelling_joints", "painful_walking"]
(vertigo) Paroymsal Positional Vertigo	["headache", "nausea", "spinning_movements", "loss_of_balance", "unsteadiness"]

Figure 2

By utilizing Jupyter Notebooks, allowing for organized code structure, a fine-tuning script is able to be created. The chosen model(s) to fine tune are the GPT-2 Small 124 million parameter model [19] and the Llama3-OpenBioLLM 8 billion parameter model [18]. The reason for two chosen LLMs is because of the necessity to ensure progress and results of the fine tuning process. The GPT-2 LLM acts as the test model, initially being fine tuned, and the Llama3 LLM will then be fine tuned depending on the test LLM's results. When creating the fine tuning script for GPT-2, it is required to first preprocess the converted dataset; special characters within the dataset –underscores– must be filtered out, in order to prevent confusion when processing tokens to the LLM. As a result, a simple preprocessing function is made to filter out the underscores and create a sequence replicating an English sentence. After successfully preprocessing the dataset, tokenization of each sequence must be done in order to be able process each token's word-embedding-vector [6]. This is done through the tokenizer class of the Hugging Face Transformer library. Figure 3 displays simple code to be able to do so.

Figure 3

The next step after tokenizing data is fine-tuning the GPT-2 model. When establishing basic training arguments and hyperparameters, as shown in figure 4, it is necessary to take into great account the hardware capabilities of a machine.

Figure 4

Once doing so, the fine tuning process is conducted by calling the train() function of the Trainer class. On completion of the process, quantitative results signified optimistic outcomes, with a

loss of ~0.2. However, upon testing the LLM's abilities within a conversational environment, several issues had become apparent. Before finalizing the preprocessing function, I had initially not properly filtered and structured the dataset in an organized manner, causing overfitting along with grammatical issues. Even though grammatical issues persisted with each fine tune, such an effect is inevitable due to the LLM's small parameter size. The first fine tune resulted in no prediction of illnesses when tested on an input of symptoms.

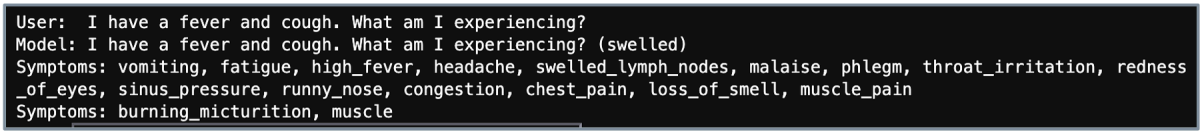

Figure 5

Instead, several repetitions of related symptoms were outlined, as shown in figure 5. After adjusting the preprocessing function to better filter the dataset, another fine tune was conducted: predictions become more apparent. However, the prediction was not reputable, suggesting an extreme case from the given symptoms, as shown in figure 6.

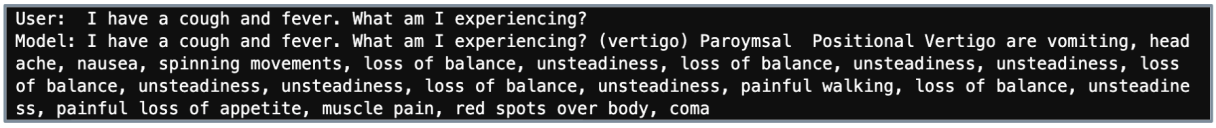

Figure 6

Upon further adjustment of the preprocessing function, a more effective structure of data was achieved, allowing for a more optimistic output from the LLM after being fine tuned another time.

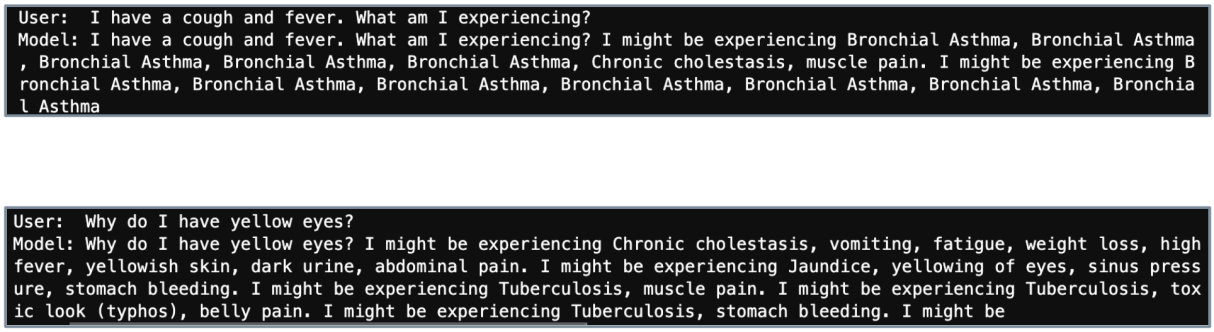

Figure 7

Predictions of illnesses became more reputable and overfitting had not occurred when testing several inputs, as shown in figure 7. Even with the unresolved grammatical errors, the GPT-2 model had achieved great results when fine tuned.

While such preprocessing and tokenization of the GPT-2 LLM is effective, the same methodology is not as effective for the Llama3 LLM. The reason for such a case is because of Llama models generally being trained/fine-tuned more effectively on data simulating the LLM's intended purpose. With this, the preprocessing function was altered to simulate my intended use of having the LLM predict illnesses within a conversational environment. The tokenization process is conducted in the same manner as the GPT-2 LLM, but instead utilizing the Llama3 OpenBioLLM tokenizer rather than the GPT-2 tokenizer, as shown in figure 8.

```
def initialize(ex):
    disease = ex["disease"]
    symptoms = ex["symptoms"]
    eng_symptoms = []

    for symptom in symptoms:

        if "_" in symptom:
            symptom = symptom.replace("_", " ")

        eng_symptoms.append(symptom)

    csv_symptoms = ", ".join(eng_symptoms)
    ex["text"] = f"<|user|>\nI have {csv_symptoms}. What disease might I have?\n<|assistant|>\nYou might have {dise
    return ex

dataset = dataset.map(initialize)
```

```
def tokenize(batch):
    ret = tokenizer(batch["text"], padding="max_length", truncation=True, max_length=256)
    ret["labels"] = ret["input_ids"].copy()

    return ret
```

Figure 8

When again establishing proper training arguments and hyperparameters, one final step must be conducted before tuning the LLM. Because of the vast difference in parameter size between the Llama3 and GPT-2 models, an optimization technique must be employed to satisfy computational requirements. By utilizing the technique of 4-bit quantization, along with low rank adapters (LoRA) the model is then able to be fine tuned on my personal machine. This optimization greatly reduces RAM requirements by representing the LLM's weights in 4-bit floats rather than 32-bit floats and injecting 8 by 8 matrices to be adjusted during the process, in place of the weights, as shown in figure 9.

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

bnb = BitsAndBytesConfig(load_in_4bit=True)

model = AutoModelForCausalLM.from_pretrained(
    "aaditya/Llama3-OpenBioLLM-8B",
    quantization_config=bnb,
    device_map="auto"
)

model = get_peft_model(model, lora_config)
```

Figure 9

Being efficient, loss is inevitably slightly greater than it would be from a full fine tune. The fine tuning process is conducted in the same fashion as the GPT-2 model. Results produced an error of ~0.021, significantly lower than the GPT-2 LLM's error and suggesting the prevention of overfitting as well as bias. Testing the LLM's abilities within a conversation environment replicating that of GPT-2's, the Llama3 LLM predicted highly reputable illness with no grammatical errors, as shown in figure 10.

```
User:  I have a fever and cough. What am I experiencing?
Model: I have a fever and cough. What am I experiencing? A) Influenza B) Pneumonia C) Bronchitis D) Asthma
```

Figure 10

A limitation of this result, however, includes the issue of not being able to eliminate uncertainty. Even though successfully providing possible illnesses, the LLM was still unable to narrow down the prediction, creating uncertainty. Such an issue is not able to be resolved, unfortunately, due to

the many similarities between common illnesses and their associated symptoms. The successful fine tuning process of the Llama3 LLM then allowed for the creation of an MCP Server.

Creating the MCP Server

An MCP server of an AI Agent is what provides the LLM access to a list of executable commands/actions, allowing it to achieve its intended goal. Such an implementation within healthcare is novel, yet to be perfectly utilized because of the many problematic circumstances that may arise and create danger. The scope of this project utilizes MCP to create a server that provides access to tools that find medication, treatment, and send information to pharmacies/those who may help the user. The universal MCP and Asyncio libraries allow for its creation, autonomously functioning when the AI Agent is used. By requesting actions to the server, the MCP Client allows the LLM to conduct actions through tools from the server. The server contains different addresses, defining each action and how it is used. At the "list_tools()" address, actions are defined –descriptions, required inputs, and its name, as shown in figure 11.

```python
@app.list_tools()
async def tools() -> list[Tool]:
    return [
        Tool(
            name = "diagnosis", # change/add tool to send medicine info, not diagnosis
            description = "Predicts possible illness from symptoms",
            inputSchema = {
                "type": "object",
                "properties": {
                    "symptoms": {
                        "type": "string",
                        "description": "symptoms' description"
                    }
                },
                "required": ["symptoms"]
            }
        ),
```

Figure 11

Once defining a tool, the logic behind how it is used is programmed under the “call_tool()” address, as shown in figure 12.

```python
@app.call_tool()
async def use_tool(name: str, args: dict) -> list[TextContent]:

    if name == "diagnosis":

        symptoms = args["symptoms"]
        prompt = f"<|user|>\nI have {symptoms}. What disease might I have?\n<|assistant|>\n"
        inputs = tokenizer(prompt, return_tensors="pt").to("cpu")

        with torch.no_grad():
            output = model.generate(**inputs, max_new_tokens = 100)

        response = tokenizer.decode(outputs[0], skip_special_tokens=True).split("<|assistant|>")[-1].strip()
        # response = "You might have Influenza."

        return[TextContent(type="text", text=response)]
```

Figure 12

Because the AI Agent involves handling sensitive information when sending/receiving user information, it is required to do so through an access token, granting access upon authorization. The AI Agent sends information through Gmail, utilizing its API to send information on behalf of the user. Such a token is enabled in Google Workspace after defining its purpose and entrepreneurial information.

Other than handling sensitive information, the MCP server finds proper medication and treatment of a user’s predicted illness. This is done through parsing the government Medline Plus database of diseases and information, and extracting what is needed. With the BeautifulSoup library, it becomes possible to do so through searching a web-page’s HTML layout to determine what must be extracted. Because of the database’s consistent page layouts, parsing most information becomes less tedious. After creating the server, an MCP Client is needed to connect the MCP server and LLM. Claude Desktop allows for the connection of a locally run server through a JSON configuration defining its communication with the server, as shown in figure 13.

```
{
  "mcpServers": {
    "disease-agent": {
      "command": "/opt/homebrew/Caskroom/miniconda/base/envs/new_env/bin/python3",
      "args": [
        "/Users/dennis/Desktop/Research/Tensorflow/Mcp_server.py"
      ]
    }
  },
```

Figure 13

The configuration is added within the MCP Client's "config.json" file, as shown in figure 14.

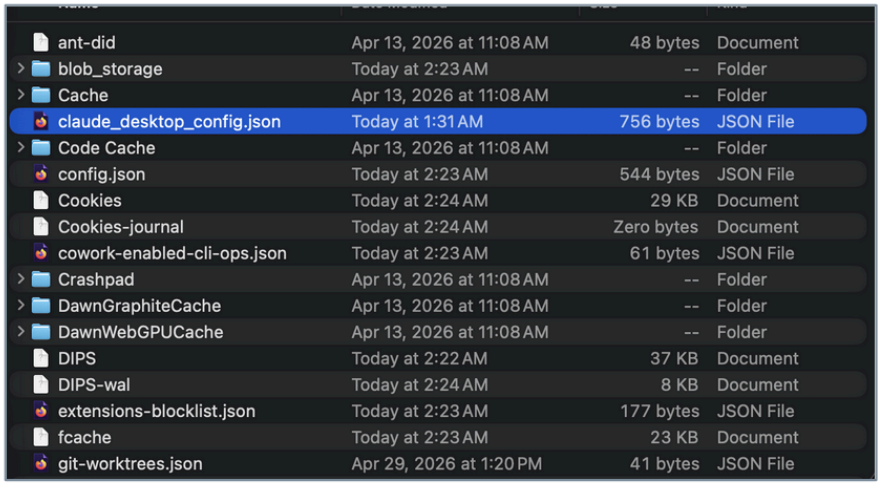

Figure 14

Once connected, the MCP server will autonomously run and the LLM will gain access to its tools, successfully creating an AI Agent.

Conclusion

Within this project, a considerable amount of refinements and limitations still remain to be addressed. While the Llama3 LLM successfully is able to diagnose common illnesses, uncertainty is still present because of its output prediction of multiple illnesses. Also, an

appropriate user interface was not yet implemented for the Agent, as Claude's ubiquitous interface was used because of its role of being the client. This limitation will be addressed in the future, along with the broader expansion of the Agent's specialization; accounting for rare illnesses would further improve its utilization within healthcare.

However, this AI Agent was able to successfully function within a conversational environment. Other models, such as Buoy AI [2], GPT 4.0, and DxGPT [1] contain user-input limitations addressed within this project. Also, Utilizing cloud services with a larger model could further improve accuracy within the healthcare domain. Furthermore, the Claude Desktop MCP Client was utilized because of its compatibility with MCP through JSON configuration. For a completely novel innovation, creating an original MCP Client would be necessary through the creation of an AI Agent Framework. All unaddressed aspects of the AI Agent are soon to be refined, along with its overall improvement.

Works Cited

[1] Alvarez-Estape, Marina, et al. "Evaluation of the Clinical Utility of DxGPT, a GPT-4 Based Large Language Model, through an Analysis of Diagnostic Accuracy and User Experience." *medRxiv*, 26 July 2024, www.medrxiv.org/content/10.1101/2024.07.23.24310847v1.

[2] Buoy Health. *Buoy Health*, www.buoyhealth.com. Accessed 1 June 2026.

[3] Centers for Medicare & Medicaid Services. "National Health Expenditure Data: Historical." *CMS.gov*, U.S. Department of Health & Human Services, 14 Jan. 2026, www.cms.gov/data-research/statistics-trends-and-reports/national-health-expenditure-data/historical.

[4] DeepSeek AI. "DeepSeek-V3.2-Exp." *Hugging Face*, huggingface.co/deepseek-ai/DeepSeek-V3.2-Exp. Accessed 1 June 2026.

[5] DelveInsight. "Top 10 Artificial Intelligence-Based Healthcare Mobile Apps." *DelveInsight Business Research*, 17 May 2023, www.delveinsight.com/blog/top-artificial-intelligence-based-healthcare-mobile-apps.

[6] GeeksforGeeks. "Tokenization in NLP." *GeeksforGeeks*, www.geeksforgeeks.org/nlp/nlp-how-tokenizing-text-sentence-words-works/. Accessed 1 June 2026.

[7] HuggingFaceFW. "FineWeb." *Hugging Face*, huggingface.co/datasets/HuggingFaceFW/fineweb. Accessed 1 June 2026.

[8] Merritt, Rick. "What Is a Transformer Model?" *NVIDIA Blog*, NVIDIA Corporation, 25 Mar. 2022, blogs.nvidia.com/blog/what-is-a-transformer-model/.

[9] Model Context Protocol. "Model Context Protocol Servers." *GitHub*, github.com/modelcontextprotocol/servers. Accessed 1 June 2026.

[10] Rahman, Md. Ashrafur, et al. "Impact of Artificial Intelligence (AI) Technology in Healthcare Sector: A Critical Evaluation of Both Sides of the Coin." Clinical Pathology, vol. 17, 2024, doi.org/10.1177/2632010X241226887.

[11] Raw, Nathan. "Huggingface-Datasets-Converter: Scripts to Convert Datasets from Various Sources to Hugging Face Datasets." GitHub, github.com/nateraw/huggingface-datasets-converter. Accessed 1 June 2026.

[12] Sciforce. "Step-by-Step Guide to Creating Your Own Large Language Model." Medium, medium.com/sciforce/step-by-step-guide-to-your-own-large-language-model-2b3fed6422d0. Accessed 1 June 2026.

[13] Shieh, Allen, et al. "Assessing ChatGPT 4.0's Test Performance and Clinical Diagnostic Accuracy on USMLE STEP 2 CK and Clinical Case Reports." Scientific Reports, vol. 14, 2024, article no. 9330, doi.org/10.1038/s41598-024-58760-x.

[14] Taware, Suvarna. "DeepSeek - Step by Step Guide." Medium, 29 Jan. 2025, medium.com/@suvarnaptaware/deepseek-step-by-step-guide-48c8fe3be7db.

[15] "What Is the Model Context Protocol (MCP)?" Model Context Protocol, modelcontextprotocol.io/docs/getting-started/intro. Accessed 1 June 2026.

[16] Zheng, Choong Qian. "Disease and Symptoms Dataset." Kaggle, www.kaggle.com/datasets/choongqianzheng/disease-and-symptoms-dataset. Accessed 1 June 2026.

[17] Tislin, Dennis. “Disease_agent.” Github, https://github.com/MagazinePuma/disease_agent. Accessed 1 June 2026.

[18] Aaditya. "Llama3-OpenBioLLM-8B." Hugging Face, 2024, huggingface.co/aaditya/Llama3-OpenBioLLM-8B. Accessed June 1 2026.

[19] Openai-community. "gpt2." Hugging Face, 2019, huggingface.co/openai-community/gpt2. Accessed June 1 2026.